

АННОТАЦИЯ

Настоящий документ является индивидуальной частью пояснительной записки к курсовому проекту «Платформа для no-code автоматизации бизнес-процессов». Документ содержит обоснование технических решений, принятых при разработке серверной части системы: выбор архитектуры, алгоритм исполнения workflow-графа, реализация Docker-песочницы, версионирование, content-addressed-хранение в MinIO, схема базы данных и REST API.

Настоящий документ разработан в соответствии с ГОСТ 19.404-79.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы	4
1.2. Краткая характеристика области применения	4
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ	5
2.1. Функциональное назначение	5
2.2. Эксплуатационное назначение	6
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	7
3.1. Постановка задачи	7
3.2. Описание алгоритма и функционирования серверной части	8
3.2.1. Архитектура серверной части	8
3.2.2. Технологический стек серверной части	11
3.2.3. Алгоритм исполнения workflow-графа	12
3.2.4. Изолированное исполнение пользовательского кода в Docker-песочнице	16
3.2.5. Версионирование workflow и снапшоты	18
3.2.6. Контент-адресуемое хранение конфигураций узлов	20
3.2.7. Трансляция событий исполнения по WebSocket	22
3.2.8. Система триггеров запуска workflow	23
3.2.9. А/В-аналитика workflow и статистическое тестирование	25
3.2.10. Стратегия автоматического тестирования и непрерывная интеграция	26
3.2.11. Схема реляционной базы данных	28
3.2.12. Программный интерфейс REST API	32
3.3. Описание и обоснование выбора метода организации входных и выходных данных ...	35
3.3.1. Описание метода организации входных данных	35
3.3.2. Описание метода организации выходных данных	35
3.3.3. Обоснование выбора метода	35
3.4. Требования к составу и параметрам технических средств	36
4. ОЖИДАЕМЫЕ КАЧЕСТВЕННЫЕ И КОЛИЧЕСТВЕННЫЕ ПОКАЗАТЕЛИ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование темы разработки — «Платформа для no-code автоматизации бизнес-процессов».

Наименование темы разработки на английском языке — «No-code business process automation platform».

Условное обозначение — «FluxPilot».

Настоящий документ является индивидуальной частью пояснительной записки к командной курсовой работе и относится к серверной части программного продукта, разрабатываемой студентом группы БДРИП242 Клушиным А.А.

1.2. Краткая характеристика области применения

Серверная часть программного продукта «Платформа для no-code автоматизации бизнес-процессов» предназначена для исполнения пользовательских workflow, представленных в виде направленных ациклических графов, и для предоставления клиентской части программного интерфейса в формате REST и WebSocket. Серверная часть обеспечивает приём конфигураций workflow от клиентской части, их хранение в реляционной базе данных PostgreSQL и объектном хранилище MinIO с использованием контент-адресуемой схемы, параллельное исполнение графов с учётом топологического порядка и изоляцией сбоев независимых ветвей, а также трансляцию событий исполнения клиентской части в режиме реального времени.

Серверная часть является составной частью единого программного продукта и не предполагает самостоятельного использования вне контекста клиентской части, реализуемой в индивидуальной части курсовой работы соавтора (студента группы БДРИП241 Гаврилина С.В.). Дополнительно к серверной части могут обращаться внешние информационные системы через входящие webhook-запросы, инициирующие запуск workflow.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Серверная часть программного продукта обеспечивает выполнение одиннадцати из четырнадцати функциональных требований общей части технического задания (RU.17701729.09.03-04 ТЗ 01-1, раздел 4.1.1, требования 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14). Полный перечень индивидуальных функциональных требований приведён в индивидуальной части технического задания (RU.17701729.09.03-04 ТЗ 03-1, раздел 2.1).

Серверная часть обеспечивает выполнение следующих основных функций. Серверная часть реализует исполнители узлов десяти типов: HTTP-узел; пять узлов обработки потоков данных (фильтрация, преобразование, агрегация, обход с побочным эффектом, разворачивание вложенных списков); узел исполнения пользовательского Python-кода; узел исполнения пользовательского JavaScript-кода; узел разделения потока на ветви по стратегии для проведения А/В-экспериментов; узел слияния ветвей А/В-эксперимента; внутренний исполнитель webhook-триггеров.

Серверная часть реализует ядро исполнения workflow с топологической сортировкой графа по алгоритму Кана, параллельным исполнением независимых ветвей через интерфейс `CompletableFuture` в выделенном пуле потоков фиксированного размера и изоляцией сбоя одной ветви от параллельных. Серверная часть реализует изолированное исполнение пользовательского кода в одноразовых Docker-контейнерах с системными ограничениями объёма оперативной памяти, процессорного времени, доступа к сети, режима файловой системы, набора Linux capabilities и количества процессов. Серверная часть реализует двухуровневую схему версионирования workflow с автоматическим созданием ревизии при каждом сохранении и поддержкой именованных версий, а также независимый механизм именованных снапшотов уровня документа.

Серверная часть реализует контент-адресуемое хранение конфигураций узлов в объектном хранилище MinIO с дедупликацией по криптографическому хешу SHA-256, что обеспечивает однократное хранение одинаковых конфигураций между ревизиями и версиями workflow. Серверная часть реализует трансляцию событий жизненного цикла исполнения в WebSocket-топик `/topic/workflows/{workflowId}/graph` с использованием протокола STOMP поверх SockJS-транспорта. Серверная часть реализует три типа триггеров запуска workflow: ручной запуск через REST-эндпойнт, scheduler-триггер с поддержкой cron-выражений и горячей перезагрузкой распи-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

сания при изменении конфигурации, webhook-триггер со сгенерированным уникальным токеном и приёмом произвольного JSON-тела в качестве входа workflow.

Серверная часть реализует серверный модуль продуктовой А/В-аналитики, включающий агрегацию просмотров и конверсий по вариантам эксперимента, расчёт доверительных интервалов и статистическую проверку значимости разности конверсий методом z-теста двух пропорций. Серверная часть реализует автоматическую генерацию документации REST API на базе библиотеки springdoc-openapi с публикацией спецификации в формате OpenAPI 3.0 и интерактивного интерфейса Swagger UI. Серверная часть реализует процесс непрерывной интеграции на базе сервиса GitHub Actions, включающий автоматический запуск модульных и интеграционных тестов на каждый pull request, сборку Docker-образов серверной и клиентской частей с публикацией в GitHub Container Registry и автоматическое обновление SVG-бейджа покрытия по JaCoCo.

2.2. Эксплуатационное назначение

Серверная часть предназначена для развёртывания на сервере под управлением операционной системы семейства Linux (Ubuntu 22.04 LTS и совместимые дистрибутивы) либо в составе оркестрированной среды Docker-контейнеров. Серверная часть обслуживает запросы клиентской части по протоколу HTTP (REST API на порту 8080) и WebSocket (STOMP-эндпойнт /v1/ws на том же порту), а также входящие webhook-запросы внешних информационных систем. Эксплуатация серверной части осуществляется системным администратором, обладающим базовыми навыками работы с системой контейнеризации Docker, реляционной базой данных PostgreSQL и объектным хранилищем MinIO.

Серверная часть развёрнута в продакшен-окружении и обслуживает публичную инстанцию программного продукта по адресу <https://fluxpilot.ru/>. REST API доступен по префиксу /v1; интерактивная документация Swagger UI размещена по адресу <https://fluxpilot.ru/v1/swagger-ui.html>. Развёртывание выполнено в соответствии с конфигурацией `deploy/docker-compose.prod.yml` и включает контейнер серверной части, развёрнутый за обратным прокси Nginx с TLS-сертификатом, контейнер PostgreSQL 16, контейнер MinIO и контейнер docker-socket-proxy для безопасного доступа серверной части к docker daemon хоста.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи

Серверная часть программного продукта должна обеспечивать выполнение одиннадцати из четырнадцати функциональных требований общей части технического задания и сопровождаться комплексом нефункциональных свойств. Серверная часть должна реализовывать движок исполнения workflow, представленных в виде направленных ациклических графов, с поддержкой топологической сортировки графа, параллельного исполнения независимых ветвей в выделенном пуле потоков и изоляции сбоев одной ветви от параллельных. Серверная часть должна предоставлять REST-интерфейс для управления workflow, версиями, снапшотами, запусками, триггерами и сводными показателями A/B-аналитики, а также WebSocket-канал для трансляции событий жизненного цикла исполнения клиентской части в режиме реального времени.

Серверная часть должна обеспечивать изолированное исполнение пользовательского Python- и JavaScript-кода в одноразовых Docker-контейнерах с системными ограничениями памяти, процессорного времени, доступа к сети, файловой системе и количеству процессов. Серверная часть должна поддерживать двухуровневую схему версионирования workflow с возможностью отката к ранее сохранённой ревизии или именованной версии, а также механизм именованных снапшотов уровня документа. Серверная часть должна обеспечивать контент-адресуемое хранение конфигураций узлов в объектном хранилище MinIO с дедупликацией по криптографическому хешу содержимого. Серверная часть должна обрабатывать три типа триггеров запуска workflow — ручной, scheduler по cron-выражению, webhook со сгенерированным токеном. Серверная часть должна предоставлять автоматически генерируемую документацию REST API в формате OpenAPI 3.0 с интерактивным интерфейсом Swagger UI.

Технологические ограничения зафиксированы в общей части технического задания и включают язык программирования Kotlin на платформе JVM, каркас приложения Spring Boot, реляционную базу данных PostgreSQL, объектное хранилище MinIO, миграции Liquibase, библиотеку автоматической документации springdoc-openapi, систему контейнеризации Docker и непрерывную интеграцию на базе GitHub Actions.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.2. Описание алгоритма и функционирования серверной части

В настоящем разделе приведены описания основных архитектурных и алгоритмических решений серверной части программного продукта. Описание сгруппировано по подразделам, каждый из которых посвящён отдельной подсистеме или ключевому алгоритму.

3.2.1. Архитектура серверной части

Серверная часть программного продукта реализована в виде модульного монолита. Программа представляет собой одно Spring Boot приложение, упакованное в исполняемый JAR-файл, с внутренним разделением исходного кода на изолированные пакеты с однонаправленными зависимостями. Такой подход обеспечивает читаемость и тестируемость кода на уровне отдельных подсистем, единую транзакционную границу при работе с реляционной базой данных и одновременно сохраняет возможность последующего вынесения отдельных подсистем в самостоятельные сервисы без существенной переработки кода.

Альтернативный подход в виде классического одиночного монолита без явного разделения на модули был отвергнут как ведущий к быстрому росту неявных зависимостей между функциональными областями и затрудняющий рефакторинг при увеличении кодовой базы. Микросервисная архитектура с выделением каждой подсистемы в самостоятельный сервис с собственной базой данных была признана избыточной для стадии MVP курсовой работы, поскольку требует значительного объёма инфраструктурной обвязки при незначительном выигрыше для текущего уровня нагрузки.

Исходный код серверной части размещён в корневом пакете `ru.startem.aelevena` и разделён на одиннадцать пакетов. Состав и назначение пакетов приведены в таблице 1.

Пакет	Назначение и состав
api	REST-контроллеры (<code>WorkflowsController</code> , <code>WorkflowVersionsController</code> , <code>WorkflowSnapshotsController</code> , <code>RunsController</code> , <code>TriggersController</code> , <code>AbAnalyticsController</code>), глобальный обработчик исключений <code>RestExceptionHandler</code> , типизированная иерархия исключений <code>ApiExceptions</code> и подпакет <code>api.dto</code> с типами данных запросов и ответов
workflow	Доменный сервис <code>WorkflowService</code> , репозитории <code>CRUD-операций</code> <code>WorkflowsRepository</code> , <code>WorkflowRevisionRepository</code> ,

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Пакет	Назначение и состав
	WorkflowVersionRepository, WorkflowSnapshotRepository, доменные модели в подпакетах workflow.model и workflow.persistence
run	Сервис исполнения WorkflowExecutionService, сервис постановки в очередь RunEnqueueService, сервис чтения RunQueryService, конфигурация пула потоков ExecutionConfig, репозитории состояния WorkflowRunRepository, NodeRunRepository
executor	Интерфейс NodeExecutor, реестр исполнителей NodeExecutorRegistry, общий движок Docker-песочницы ContainerSandboxRunner, предзагрузчик базовых образов SandboxImageWarmer, реализации исполнителей HttpNodeExecutor, PythonNodeExecutor, JavaScriptNodeExecutor, DataflowNodeExecutors, TriggerWebhookExecutor, BranchSplitNodeExecutor, BranchSplitStrategies, SplitEnvelope, BranchMergeNodeExecutor
triggers	Доменный сервис TriggerService, планировщик cron-задач TriggerScheduler, конфигурация SchedulerConfig, репозиторий TriggersRepository
blob	Сервис контент-адресуемого хранения BlobService и репозиторий индекса хешей BlobIndexRepository
ws	WebSocket-контроллер WorkflowWsController, слушатель доменных событий GraphBroadcastListener, конфигурация WebSocket-брокера WebSocketConfig
analytics	Сервис продуктовой аналитики AbAnalyticsService, репозиторий чтения variant-rows и excluded count AbAnalyticsRepository, статистический тест разности конверсий StatTest, типы данных ответов AbAnalyticsDtos
seed	Сервис посева демонстрационных workflow DemoWorkflowSeeder, описание плана DemoWorkflowPlan и подпакет seed.plans с готовыми планами
config	Конфигурационные бины Spring OpenApiConfig, JacksonConfig, S3Config, S3Properties, WebMvcConfig

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Пакет	Назначение и состав
util	Утилиты CanonicalJson (приведение JSON-дерева к канонической форме) и Hashing (вычисление SHA-256)

Таблица 1. Пакеты серверной части

Зависимости между пакетами имеют строго однонаправленный характер. Пакеты `api` и `api.dto` зависят от `workflow`, `run`, `triggers`, `analytics`. Пакет `run` зависит от `executor`, `blob`, `ws`. Пакет `executor` зависит от `blob`. Обратных зависимостей не предусмотрено. Соблюдение направленности зависимостей контролируется при просмотре изменений кода.

Контейнерное представление всей системы с указанием взаимодействий серверной части с инфраструктурными компонентами (PostgreSQL, MinIO, docker-socket-proxy, песочница), клиентской частью, обратным прокси Nginx и внешними источниками webhook-запросов приведено на рисунке 1.

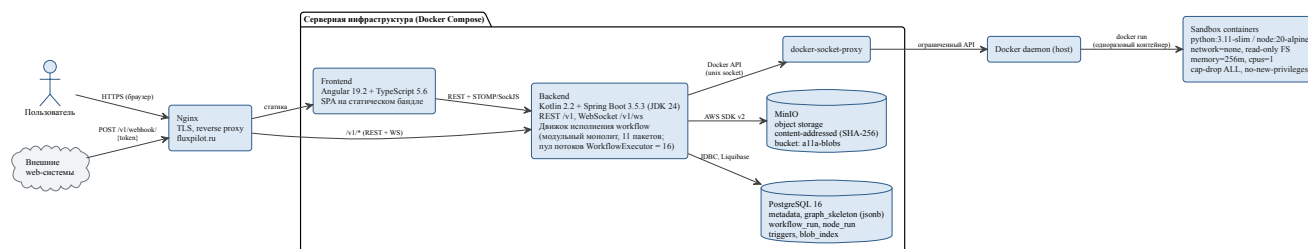


Рис. 1. Контейнерная архитектура программного продукта (нотация С4, уровень контейнеров)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

В целях развития системы предусмотрена возможность горизонтального масштабирования серверной части на несколько реплик без введения дополнительных инфраструктурных сервисов: реляционная база данных PostgreSQL используется в роли распределённой очереди задач (claim записей таблицы `workflow_run` через `SELECT ... FOR UPDATE SKIP LOCKED`, lease-механизм с полями `lease_owner`, `lease_until` и `last_heartbeat` для перезапуска незавершённых runs при отказе реплики, `pg_try_advisory_lock` для выбора единственного лидера cron-планировщика, `LISTEN/NOTIFY` на канале `ws_broadcast` для трансляции WebSocket-событий между репликами). Описанный подход позволяет масштабировать развёртывание командой `docker compose --scale backend=N` после применения соответствующей миграции схемы. Аналогичная возможность предусмотрена для пакета `blob` с введением собственного жизненного цикла объектов (TTL для неиспользуемых `blob` и фоновая компактификация). В рамках курсовой работы данные расширения не реализуются.

3.2.2. Технологический стек серверной части

Серверная часть программного продукта реализована на языке программирования Kotlin версии 2.2.21 с использованием каркаса Spring Boot версии 3.5.3, исполняемого на платформе JDK 24. Выбор языка обусловлен сочетанием строгой статической типизации с поддержкой null-safety на уровне системы типов, эффективной модели многопоточности через пулы потоков JVM, наличием зрелой экосистемы готовых модулей для решения типовых задач серверного приложения и компактностью синтаксиса по сравнению с Java.

Альтернативные варианты языка и каркаса (Go со стандартной библиотекой и сторонними маршрутизаторами, Node.js на TypeScript с каркасом NestJS, Python с каркасом FastAPI) были признаны менее подходящими по совокупности факторов. Go обладает менее развитой экосистемой готовых решений уровня enterprise и не предоставляет готовой реализации STOMP-WebSocket. Node.js использует однопоточный event loop, что ограничивает эффективность параллельной обработки CPU-интенсивных операций; типизация TypeScript стирается при сборке и не обеспечивает гарантий на этапе исполнения. Python ограничен глобальным интерпретатором GIL и динамической типизацией; дополнительный аргумент против Python состоит в том, что пользовательский Python-код требует исполнения в изолированной среде, а совпадение языка серверной части и пользовательского кода повышает риски при выходе из песочницы.

В состав используемых модулей каркаса Spring входят Spring Web (REST-контроллеры), Spring WebSocket (STOMP-эндпойнт), Spring JDBC (взаимодействие с PostgreSQL без использо-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

вания ORM), Spring Validation (декларативная валидация DTO), Spring Scheduler (планировщик cron-задач). Для автоматической генерации документации REST API используется библиотека springdoc-openapi версии 2.7.0, публикующая спецификацию OpenAPI 3.0 и интерактивный интерфейс Swagger UI по адресу /v1/swagger-ui.html. Сериализация JSON осуществляется библиотекой Jackson с подключённым модулем Kotlin. Взаимодействие с объектным хранилищем MinIO осуществляется через AWS SDK for Java версии v2. Миграции схемы реляционной базы данных управляются библиотекой Liquibase с описанием changeset-ов в формате YAML. Тестирование выполняется библиотекой JUnit Jupiter версии 5; для интеграционных тестов с реальными контейнерами PostgreSQL и MinIO применяется библиотека Testcontainers версии 1.21.1.

Сборка серверной части осуществляется системой Apache Maven версии 3.9 с использованием Spring Boot Maven Plugin для упаковки в исполняемый fat-JAR. Развёртывание выполняется в Docker-контейнере на базе образа eclipse-temurin:24-jre. Непрерывная интеграция настроена в GitHub Actions и запускается на каждый pull request и push в основную ветку.

3.2.3. Алгоритм исполнения workflow-графа

Workflow в системе представляется направленным ациклическим графом, вершины которого соответствуют узлам обработки, а рёбра — связям передачи данных между узлами. Ядро исполнения должно обеспечивать обработку графа с соблюдением порядка зависимостей, параллельным выполнением независимых ветвей и изоляцией сбоев одной ветви от параллельных, а также валидацию графа на отсутствие циклов до начала исполнения.

В качестве алгоритма исполнения применяется комбинированный подход: на этапе подготовки выполняется топологическая сортировка графа по алгоритму Кана, одновременно служащая проверкой ацикличности, после чего строится цепочка экземпляров CompletableFuture для каждого узла и запускается event-driven исполнение в выделенном пуле потоков. Альтернативные подходы — последовательный обход в глубину или ширину, а также уровневое исполнение после топологической сортировки — были отвергнуты. Последовательный обход не обеспечивает параллелизма и неприемлем для workflow с длительными HTTP-узлами. Уровневое исполнение вводит искусственную синхронизацию на границах уровней: при наличии в уровне одного длительного узла остальные узлы следующего уровня вынуждены ожидать его завершения, хотя могли бы стартовать сразу после готовности своих непосредственных предков.

Ядро исполнения реализовано в классе WorkflowExecutionService пакета ru.startem.aelevena.run в методе executeWorkflow(workflowRunId: Long). Алгоритм работает в

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

следующей последовательности. Из таблицы `workflow_run` извлекается запись запуска, содержащая идентификаторы `workflow` и связанной ревизии; из таблицы `workflow_revision` извлекается поле `graph_skeleton` типа `jsonb`, содержащее полную структуру графа в виде списка узлов с их типами и хешами конфигураций, а также списка рёбер. Поле `graph_skeleton` десериализуется в доменные объекты узлов и рёбер; на их основе строятся словари входящих и исходящих рёбер для каждого узла.

Проверка ацикличности выполняется методом `topologicalOrder(inDegree, outgoing)` по алгоритму Кана. Словарь `in-degree` инициализируется размерами множеств входящих рёбер; в очередь помещаются узлы с нулевым `in-degree`. На каждой итерации из очереди извлекается узел, добавляется в выходной список и для каждого его потомка уменьшается счётчик входящих рёбер; при достижении нулевого значения потомок добавляется в очередь. Если по завершении алгоритма размер выходного списка меньше количества узлов графа, делается вывод о наличии цикла, запуск переводится в статус `failed` с диагностическим сообщением «Cycle detected in workflow graph» и возвращается ранний отказ без исполнения узлов.

При успешной топологической сортировке для каждого узла графа в таблицу `node_run` вставляется запись со статусом `queued`, запуск переводится в статус `running` с фиксацией времени старта. Далее в порядке топологической сортировки для каждого узла создаётся экземпляр `CompletableFuture`. Корневой узел получает уже-готовый `future`. Для узла с предками формируется составной `future` через вызов `CompletableFuture.allOf(*depFutures)`, на который методом `thenApplyAsync` навешивается асинхронная задача исполнения, передаваемая в пул потоков `workflowExecutor` (размер пула 16, конфигурируется через свойство `app.executor.pool-size`). В задаче исполнения загружается конфигурация узла из MinIO по хешу через `BlobService.getJsonTree(configHash)`, извлекается соответствующий `NodeExecutor` из реестра `NodeExecutorRegistry` по типу узла, выполняется метод `execute(input, config)`, результат сохраняется в `node_run` со статусом `success` и публикуется событие `node_exited` в WebSocket-канал. Независимые узлы исполняются одновременно по мере готовности их предков и наличия свободных потоков в пуле.

Обработка сбоев реализована через обработчик `whenComplete`. При возникновении исключения в задаче исполнения узла запись `node_run` обновляется со статусом `failed`, описание ошибки сохраняется в поле `error_message`. Все непосредственные потомки сбойного узла получают статус `skipped` с диагностическим сообщением «Dependency failed: »; статус `skipped`

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

транзитивно распространяется по всей цепочке потомков. Параллельные ветви, не зависящие от сбойного узла, продолжают исполнение, что обеспечивает заявленную отказоустойчивость. По завершении всех future-цепочек определяется итоговый статус `workflow_run`: `success` при успешном завершении всех узлов или `failed` при наличии хотя бы одного сбойного или пропущенного узла. На протяжении всего исполнения слушатель `GraphBroadcastListener` транслирует доменные события (`workflow_started`, `node_reached`, `node_action`, `node_exited`, `workflow_finished`) в STOMP-топик `/topic/workflows/{workflowId}/graph`, на который подписана клиентская часть для отображения хода исполнения в редакторе.

Корректность алгоритма подтверждается интеграционным тестом `MvpIntegrationTests.testParallelBranchesWithFailure()`, поднимающим реальные контейнеры PostgreSQL и MinIO через Testcontainers, создающим workflow с двумя параллельными ветвями (HTTP-узел с заведомо невалидным URL и цепочка из двух Code Node-узлов), запускающим исполнение и проверяющим, что вторая ветвь успешно завершилась, несмотря на сбой первой.

Последовательность взаимодействий участников при исполнении workflow от момента приёма триггера до публикации завершающего события в WebSocket-канал приведена на рисунке 2.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

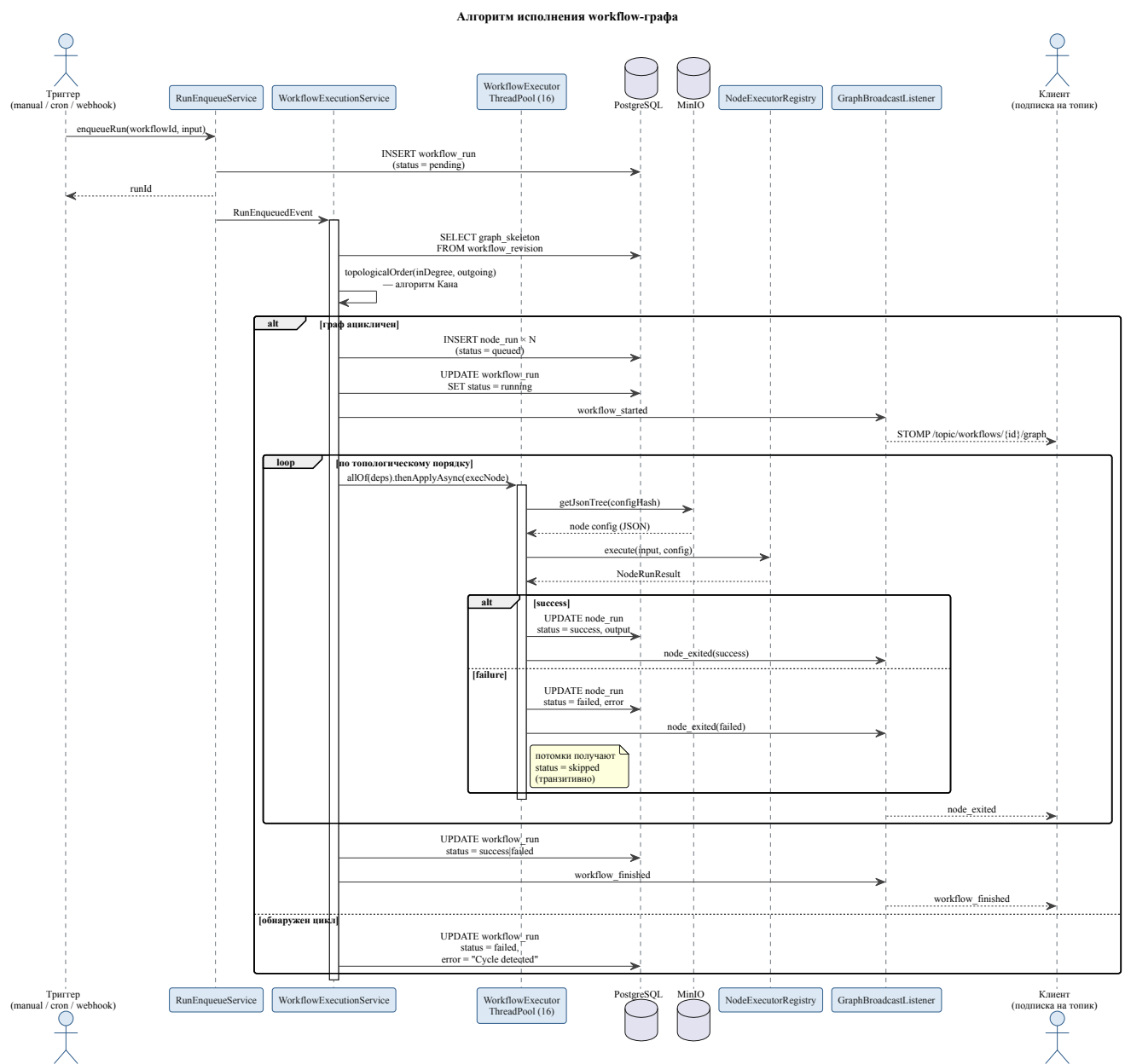


Рис. 2. Алгоритм исполнения workflow-графа (sequence-диаграмма)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Помимо штатного исполнения workflow целиком серверная часть поддерживает режим пошаговой отладки, при котором пользователь может запустить одну выбранную ноду графа с произвольным входным значением, не прогоняя предшествующие узлы. Указанный режим использует ту же реестровую инфраструктуру исполнителей и тот же механизм трансляции событий, что и штатное исполнение, и предназначен для итеративной разработки конфигурации отдельных узлов (выражений фильтрации, фрагментов пользовательского кода, шаблонов HTTP-запросов) с возможностью наблюдения входных и выходных данных без необходимости полного прогона графа. Реализация интегрирована в общий поток исполнения через специальный режим вызова с указанием идентификатора целевого узла; результат возвращается клиентской части в синхронном виде и одновременно фиксируется в истории запусков с пометкой отладочного исполнения.

В качестве направлений дальнейшего развития предусмотрено введение узлов условных переходов с дополнительным типом рёбер; циклических конструкций над входной коллекцией; горизонтального масштабирования исполнения на множество реплик серверной части с использованием реляционной базы данных PostgreSQL в качестве распределённой очереди задач (claim записей через `SELECT ... FOR UPDATE SKIP LOCKED`, lease-механизм с heartbeat для перезапуска незавершённых runs при отказе реплики, advisory lock для выбора лидера cron-планировщика, механизм LISTEN/NOTIFY для трансляции WebSocket-событий между репликами) без введения дополнительных инфраструктурных сервисов; квот пула потоков по типам триггеров для предотвращения голодания интерактивных запусков.

3.2.4. Изолированное исполнение пользовательского кода в Docker-песочнице

Узел Code Node предоставляет пользователю возможность исполнения произвольного программного кода на Python или JavaScript в составе workflow. Указанная функциональность является критической с точки зрения безопасности: пользовательский код потенциально способен выполнять произвольные системные вызовы, читать и модифицировать файлы серверной части, обращаться к локальной сети инфраструктуры, исчерпывать оперативную память и процессорное время. Серверная часть должна обеспечивать полную изоляцию пользовательского кода: отсутствие доступа к файловой системе хоста за пределами выделенной директории, отсутствие сетевого доступа, ограничения объёма оперативной памяти и процессорного времени, ограничение количества процессов, drop всех Linux capabilities, исключение возможности повышения привилегий.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

В качестве среды исполнения пользовательского кода используется одnorазовый Docker-контейнер. Альтернативные варианты — исполнение в той же JVM через библиотеку GraalVM Polyglot и запуск дочернего процесса с применением системных механизмов `rlimit` и `seccomp` — были отвергнуты. GraalVM Polyglot ограничен по совместимости с C-расширениями экосистемы Python, не предоставляет надёжных лимитов оперативной памяти на уровне отдельного контекста и не гарантирует тайм-аут по `wall-clock`. Запуск дочернего процесса с `rlimit` и `seccomp` использует общее пространство имён файловой системы и сети с родительским процессом, что требует ручной настройки `chroot` и сетевого `namespace` и сопряжено с риском ошибки конфигурации с потерей изоляции. Docker-контейнер обеспечивает максимальную изоляцию через отдельные Linux `namespace` (PID, network, mount, IPC, UTS) и готовый механизм лимитов через `cgroups`, при этом обеспечивая чистую модель «запуск — получение результата — удаление контейнера» и поддержку нескольких языков сменой базового образа.

Реализация представлена двумя специализированными исполнителями `PythonNodeExecutor` и `JavaScriptNodeExecutor`, которые делегируют запуск контейнера общему универсальному движку `ContainerSandboxRunner`. Движок инкапсулирует формирование команды Docker, управление временной директорией, передачу входных данных через стандартный поток ввода, тайм-аут исполнения, чтение стандартных потоков вывода и разбор JSON-результата. В качестве базовых образов используются `python:3.11-slim` для Python и `node:20-alpine` для JavaScript. На старте серверной части вспомогательный класс `SandboxImageWarmer` выполняет команду `docker pull` для обоих базовых образов, исключая холодный старт при первом исполнении пользовательского кода.

Алгоритм запуска контейнера включает следующие шаги. В файловой системе хоста создаётся уникальная временная директория, в которую записываются файл-обёртка `runner.py` или `runner.js` (читающая входные данные из стандартного потока ввода, вызывающая пользовательскую функцию и печатающая результат в стандартный поток вывода в формате JSON) и файл с входными данными узла. Запускается команда `docker run --rm -i --network none --read-only --tmpfs /tmp:rw,size=16m --cap-drop ALL --security-opt no-new-privileges --memory 256m --cpus 1 --pids-limit 64 -v {tmpDir}:/work:ro {base-image} {runner-command}`. Флаг `--rm` обеспечивает автоматическое удаление контейнера после завершения; `--network none` полностью отключает сетевой доступ; `--read-only` устанавливает корневую файловую систему в режим только для чтения; `--tmpfs /tmp:rw,size=16m` предоставляет 16 МБ записываемой временной

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

памяти, необходимой интерпретатору для компиляции байт-кода; `--cap-drop ALL` снимает все Linux capabilities; `--security-opt no-new-privileges` блокирует механизмы повышения привилегий; `--memory 256m` ограничивает объём оперативной памяти с принудительным завершением при превышении; `--cpus 1` ограничивает процессорное время одним ядром; `--pids-limit 64` ограничивает количество процессов и потоков; флаг `-v {tmpDir}:/work:ro` монтирует временную директорию хоста в режиме только для чтения.

Ожидание завершения процесса контейнера выполняется с тайм-аутом, складывающимся из конфигурируемого тайм-аута пользовательского кода (по умолчанию 5 секунд) и служебного overhead 30 секунд на запуск контейнера и инициализацию интерпретатора. При превышении тайм-аута контейнер принудительно завершается вызовом `process.destroyForcibly()` с возбуждением исключения `NodeTimeoutException` и диагностическим сообщением «Sandbox timeout». Стандартный поток вывода читается и разбирается как JSON, формируется результат `NodeRunResult.success(output)`. Стандартный поток ошибок читается параллельно и при ненулевом коде завершения процесса включается в сообщение об ошибке. В блоке `finally` временная директория удаляется штатной утилитой обхода файлового дерева.

Корректность работы песочницы подтверждается набором ручных и автоматических методик испытаний, описанных в индивидуальной программе и методике испытаний (RU.17701729.09.03-04 51 03-1, раздел 5.4): запуск корректного кода с возвратом JSON, попытка обращения к сети из пользовательского кода с ожидаемым отказом, попытка записи в файловую систему с ожидаемым отказом, превышение лимита оперативной памяти с ожидаемым завершением по ООМ, бесконечный цикл с ожидаемым завершением по тайм-ауту, попытка повышения привилегий через `sudo` с ожидаемым отказом.

В качестве направлений развития предусмотрены: `warm pool` предзагруженных контейнеров с поддержкой режима паузы для сокращения латентности запуска; поддержка дополнительных языков (Ruby, Lua, R) через смену базового образа; применение runtime gVisor вместо стандартного runc для дополнительного уровня изоляции через перехват системных вызовов в пользовательском пространстве; аудит системных вызовов пользовательского кода средствами eBPF для post-mortem анализа подозрительной активности.

3.2.5. Версионирование workflow и снимоты

Конфигурация workflow в редакторе подвергается частым изменениям в процессе подбора параметров узлов и проведения экспериментов. Серверная часть должна обеспечивать автомати-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ческое сохранение каждого изменения для предотвращения потери работы при сбое клиента, явное создание именованных версий для фиксации стабильных состояний workflow, возможность отката к произвольной ранее сохранённой версии или ревизии, а также схему хранения, исключаящую избыточность данных при многократном сохранении однотипных конфигураций.

В качестве схемы хранения применяются полные снимки в двухуровневой иерархии. Альтернативные варианты — хранение различий между ревизиями по модели Git и event sourcing с реконструкцией состояния через повторное проигрывание команд — были отвергнуты. Хранение различий усложняет реализацию операций чтения произвольной ревизии (требуется применение последовательности патчей от опорной точки) и затрудняет отладку. Event sourcing предполагает значительный объём инфраструктурной обвязки для поддержки операций rebase и squash и приводит к замедлению чтения при росте количества событий. Полные снимки обеспечивают чтение произвольной ревизии за константное время и упрощают аудит, а проблема избыточности хранения решается на нижнем уровне за счёт контент-адресуемого хранения конфигураций узлов с дедупликацией (см. подраздел 3.2.6).

Первый уровень иерархии — автоматические ревизии. Каждое сохранение конфигурации через REST-эндпойнт PUT /workflows/{id} создаёт новую запись в таблице workflow_revision с инкрементальным номером revision_number и полным сериализованным графом в поле graph_skeleton типа jsonb. Второй уровень — именованные версии. По явному запросу пользователя через REST-эндпойнт POST /workflows/{id}/versions с указанием тега создаётся запись в таблице workflow_version, ссылающаяся на конкретную ревизию через внешний ключ root_revision_id с ограничением ON DELETE RESTRICT. Указанное ограничение исключает удаление ревизии, на которую ссылается именованная версия, и тем самым гарантирует сохранность исторических версий при чистке устаревших ревизий.

Дополнительный третий уровень — именованные снимки уровня документа, реализованные миграцией Liquibase 005-workflow-snapshots.yaml и таблицей workflow_snapshot. В отличие от именованных версий, снимки содержат пользовательские поля имени и описания и предназначены для фиксации значимых вех развития workflow. REST-эндпойнты создания, чтения, обновления и удаления снимков реализованы в контроллере WorkflowSnapshotsController.

Доменный сервис WorkflowService реализует следующие основные операции. Операция сохранения конфигурации updateWorkflow(workflowId, request) сохраняет каждую конфигу-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

рацию узла графа в MinIO через сервис `BlobService.put(json)` с получением SHA-256-хеша, собирает структуру графа с хешами конфигураций в поле `graph_skeleton`, формирует следующий номер ревизии и вставляет новую запись в таблицу `workflow_revision`. Операция создания именованной версии `createVersion(workflowId, versionTag)` фиксирует ссылку на текущую ревизию `workflow` в таблице `workflow_version`. Операция отката `restoreVersion(workflowId, versionId)` извлекает граф из ревизии, на которую ссылается версия, и создаёт новую ревизию с тем же содержимым графа, не модифицируя существующие ревизии. Указанный подход реализует `append-only`-семантику исторических данных и обеспечивает полную трассируемость операций отката.

Иммутабельность исторических ревизий и версий обеспечена структурно. Репозитории не предоставляют методов изменения и удаления записей ревизий и версий. Ограничения внешних ключей в схеме реляционной базы данных, описанные в миграциях Liquibase, блокируют удаление ревизий, на которые имеются ссылки из именованных версий или снапшотов.

В качестве направлений развития предусмотрены: визуализация различий между двумя wybranными ревизиями в клиентской части с подсветкой добавленных, удалённых и изменённых узлов; именованные ветки `workflow` по аналогии с механизмом ветвления Git с поддержкой операции слияния; фоновая очистка ревизий по политике хранения; экспорт версии в файл формата YAML или JSON для версионирования вне серверной части.

3.2.6. Контент-адресуемое хранение конфигураций узлов

Конфигурации узлов `workflow` (в частности, конфигурации узлов Code Node с объёмными блоками пользовательского кода и HTTP-узлов с многостраничными JSON-телами) могут занимать существенный объём дискового пространства. При частом сохранении ревизий и наличии множества рабочих копий `workflow` без дополнительных мер возникает риск экспоненциального роста хранилища. Серверная часть должна исключать многократное сохранение идентичных конфигураций между ревизиями.

В качестве схемы хранения применяется контент-адресуемое хранение с использованием криптографического хеша SHA-256. Альтернативные подходы — хранение конфигураций непосредственно в `jsonb`-полях реляционной базы без дедупликации и реализация собственного расформата с внутренней дедупликацией — были отвергнуты. Первый подход неэффективен по дисковому пространству. Второй требует разработки собственного движка хранения, что выходит за рамки задач курсовой работы. Контент-адресуемое хранение обеспечивает естественную деду-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

пликацию (повторное сохранение содержательно идентичной конфигурации возвращает тот же ключ), устойчивые ссылки (ключ определяется содержимым), возможность независимой репликации хранилища и валидацию целостности при чтении за счёт повторного вычисления хеша.

Реализация представлена тремя классами. Утилита CanonicalJson приводит произвольное JSON-дерево к канонической форме: ключи объектов сортируются лексикографически, пробелы между токенами нормализуются, числовые значения приводятся к каноническому представлению. Утилита Hashing вычисляет SHA-256-хеш массива байтов через стандартный механизм MessageDigest и возвращает шестнадцатеричное представление. Сервис BlobService предоставляет два публичных метода: `put(json: JsonNode): String` для сохранения конфигурации в MinIO с возвратом хеша и `getJsonTree(hash: String): JsonNode` для чтения конфигурации по хешу с проверкой целостности.

Алгоритм сохранения включает следующие шаги. JSON-дерево приводится к канонической форме, выполняется кодирование в массив байтов в кодировке UTF-8, вычисляется SHA-256-хеш. Формируется ключ объекта в MinIO в формате `blobs/{hash}`. Выполняется запись объекта в MinIO; операция идемпотентна: повторная запись существующего объекта не приводит к ошибке и не вызывает фактической перезаписи на стороне MinIO. Запись в индексной таблице `blob_index` выполняется с предикатом `ON CONFLICT DO NOTHING`, что обеспечивает корректное поведение при конкурентной записи одинаковых конфигураций.

Алгоритм чтения включает извлечение объекта по ключу из MinIO, повторное вычисление SHA-256-хеша на полученных байтах для проверки целостности, разбор содержимого как JSON-дерева. При несовпадении вычисленного хеша с запрошенным возбуждается исключение нарушения целостности.

Корректность работы дедупликации подтверждается интеграционным тестом `MvpIntegrationTests.testBlobDeduplicationAcrossRevisions()`, создающим две ревизии workflow с одной идентичной конфигурацией узла и проверяющим, что в индексной таблице для соответствующего хеша создана единственная запись, а в MinIO по префиксу `blobs/` присутствует единственный объект.

В качестве направлений развития предусмотрены: фоновая очистка неиспользуемых объектов (объекты, на которые не ссылается ни одна актуальная ревизия) по политике TTL; мониторинг роста объёма bucket средствами Prometheus с алертированием при превышении пороговых

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

значений; шифрование объёмных конфигураций перед загрузкой в MinIO; применение алгоритма сжатия `zstd` для конфигураций большого размера.

3.2.7. Трансляция событий исполнения по WebSocket

Клиентская часть должна отображать ход исполнения `workflow` в режиме реального времени: подсвечивать узел, находящийся в исполнении, переключать визуальные состояния узлов по мере их завершения, пометать пропущенные узлы, отображать прогресс. Периодический опрос REST-эндпойнта статуса с клиентской стороны создаёт существенную нагрузку на серверную часть и реляционную базу данных и не обеспечивает требуемой латентности обновления. Серверная часть должна реализовывать `push`-механизм передачи событий клиентской части с гарантированной упорядоченностью.

В качестве протокола трансляции событий применяется WebSocket с прикладным протоколом STOMP поверх SockJS-транспорта. Альтернативные варианты — однонаправленный канал `Server-Sent Events` и `long polling` — были отвергнуты. SSE не поддерживает топиков-подписок (одно соединение обслуживает один поток) и плохо проходит через корпоративные прокси с буферизацией. `Long polling` характеризуется высокой латентностью и значительной нагрузкой на серверную часть. WebSocket с STOMP-протоколом предоставляет полнодуплексный канал с нативной поддержкой топиков `publish/subscribe`, а SockJS-транспорт обеспечивает автоматический `fallback` на `long-polling` при невозможности `WebSocket-handshake` в корпоративных сетях.

Конфигурация WebSocket-брокера реализована в классе `WebSocketConfig`, имплементирующем интерфейс `WebSocketMessageBrokerConfigurer`. Регистрируется STOMP-эндпойнт `/v1/ws` с поддержкой SockJS, настраивается встроенный `simple in-memory message broker` с префиксом топиков `/topic/`. Слушатель доменных событий `GraphBroadcastListener` подписан на механизм публикации событий Spring и обрабатывает события изменения состояния узлов и `workflow`.

Алгоритм трансляции события включает следующие шаги. При изменении статуса узла в процессе исполнения сервис `WorkflowExecutionService` создаёт доменное событие `NodeRunStatusChangedEvent` с идентификаторами `workflow` и `node_run`, предыдущим и новым статусом, выходными данными или сообщением об ошибке. Событие публикуется через стандартный `ApplicationEventPublisher` Spring. Слушатель `GraphBroadcastListener` принимает событие, формирует DTO-сообщение и публикует его в STOMP-топик `/topic/workflows/{workflowId}/graph` через `SimpMessagingTemplate`. Клиентская часть, подписанная на топик че-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

рез библиотеки `@stomp/stompjs` и `sockjs-client`, принимает сообщение и обновляет визуальное состояние узла в редакторе.

Состав транслируемых событий включает `workflow_started` (момент начала исполнения с временной меткой), `node_reached` (переход узла в статус `running`), `node_action` (промежуточные события узла, зарезервировано для `streaming`-режима), `node_exited` (финальный статус узла с выходными данными или сообщением об ошибке), `workflow_finished` (момент завершения исполнения с итоговым статусом и агрегированным результатом). Контракт событий зафиксирован в DTO-классах подпакета `api.dto.ws`. Упорядоченность сообщений обеспечивается тем, что публикация событий выполняется в потоке исполнения узла, а STOMP-брокер сериализует сообщения в порядке отправки.

В качестве направлений развития предусмотрены: аутентификация и авторизация подписки на топики с проверкой прав доступа пользователя к конкретному `workflow`; переход с встроенного `simple in-memory broker` на полноценный STOMP-брокер с кластеризацией (`RabbitMQ` или `ActiveMQ Artemis`) для поддержки множества экземпляров серверной части; батчинг частых событий с агрегацией нескольких изменений в одно сообщение при высокой интенсивности событий.

3.2.8. Система триггеров запуска `workflow`

Серверная часть должна поддерживать три типа триггеров запуска `workflow`: ручной запуск пользователем через клиентскую часть, периодический запуск по расписанию (`scheduler`), запуск при поступлении входящего HTTP-запроса (`webhook`). Триггеры должны поддерживать состояние активности (включён/выключен) без удаления из системы, обеспечивать параметризацию (`cron`-выражение для `scheduler`-триггера, генерируемый токен для `webhook`-триггера) и обновление расписания `scheduler`-триггера без перезапуска серверной части.

В качестве планировщика `scheduler`-триггеров применяется встроенный механизм `Spring TaskScheduler`. Альтернативный вариант с использованием библиотеки `Quartz Scheduler` был признан избыточным для стадии MVP, поскольку `Quartz` ориентирован на `persistent jobs` с кластерным исполнением и распределённой координацией, которые не требуются на текущем уровне нагрузки. Применение внешнего системного `cron`-сервиса с вызовом REST-эндпойнта было отвергнуто как ведущее к разрыву единого жизненного цикла триггеров между приложением и операционной системой хоста. Для `webhook`-триггеров применяется схема уникального пути с UUID-токеном в URL. Альтернативный вариант с общим путём и токеном в HTTP-заголовке был отвергнут как менее удобный для внешних систем, требующий кастомной конфигурации заголовков.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Доменный сервис `TriggerService` и репозиторий `TriggersRepository` реализуют управление триггерами. Таблица `triggers` создаётся миграцией `001-init.yaml` с расширением колонкой `node_id` в миграции `002-trigger-node-id.yaml` для возможности привязки триггера к конкретному стартовому узлу графа. Структура таблицы включает поля идентификатора, идентификатора `workflow`, типа триггера, конфигурации в формате `jsonb`, уникального токена для `webhook`-триггера, флага активности и опциональной привязки к узлу графа.

Ручной триггер реализован REST-эндпойнтом `POST /v1/workflows/{id}/runs` контроллера `RunsController`. Контроллер принимает идентификатор `workflow` и опциональное тело запроса, передаваемое в качестве входа первого узла, и вызывает `RunEnqueueService.enqueueRun(workflowId, input, triggerType)`. Сервис создаёт запись запуска со статусом `pending`, публикует доменное событие `RunEnqueuedEvent`, на которое подписан фоновый обработчик `WorkflowExecutionService.handleRunEnqueued()`, инициирующий собственно исполнение в пуле потоков.

`Scheduler`-триггер реализован классом `TriggerScheduler`. На старте серверной части обработчик с аннотацией `@PostConstruct` загружает все включённые `scheduler`-триггеры из репозитория, для каждого создаёт `CronTrigger` Spring с `cron`-выражением из конфигурации и регистрирует в `TaskScheduler` с `lambda`-обработчиком, вызывающим `RunEnqueueService.enqueueRun(...)`. При создании, обновлении или удалении триггера через REST API сервис `TriggerService` публикует событие `TriggerChangedEvent`, на которое подписан метод `TriggerScheduler.refreshTriggers()`, перезагружающий расписание без перезапуска серверной части. Поддерживается формат `cron`-выражений Spring (6 полей: секунды, минуты, часы, день месяца, месяц, день недели).

`Webhook`-триггер создаётся REST-эндпойнтом `POST /v1/workflows/{id}/triggers` с телом запроса, содержащим тип `webhook`. Сервис `TriggerService` генерирует токен `UUID.randomUUID().toString()` и сохраняет триггер в таблице с уникальным ограничением на поле токена. В ответе возвращается сгенерированный URL вида `/v1/webhook/{token}`. Приём `webhook`-запросов реализован контроллером `TriggersController` методом `handleWebhook`, который отвечает на запросы `POST /v1/webhook/{token}`, извлекает триггер по токenu, проверяет флаг активности, читает тело запроса в формате произвольного JSON и вызывает `RunEnqueueService.enqueueRun(workflowId = trigger.workflowId, input = body, triggerType = "webhook")`. Клиенту возвращается HTTP-статус 202 Accepted с идентификатором созданного

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

запуска. Токен webhook-триггера стабилен для конкретного триггера и не меняется между повторными запросами.

В качестве направлений развития предусмотрены: отложенные триггеры с возможностью отмены до момента запуска; условные триггеры с фильтрацией по содержимому входного запроса; retry-политика для webhook с поддержкой семантики идиempотентности на основе заголовка Idempotency-Key; проверка HMAC-подписи входящих webhook-запросов для защиты от спуфинга токенов.

3.2.9. А/В-аналитика workflow и статистическое тестирование

Серверная часть реализует серверный модуль продуктовой А/В-аналитики, обеспечивающий подсчёт сводных показателей экспериментов и статистическую проверку значимости разности конверсий между вариантами. Модуль аналитики работает в связке с узлами разделения и слияния ветвей графа: узел BranchSplitNodeExecutor распределяет входной поток на N вариантов согласно выбранной стратегии, узлы пользовательской ветви фиксируют просмотры и конверсии в результатах исполнения, узел BranchMergeNodeExecutor собирает результаты ветвей в единый поток, после чего сводные показатели становятся доступны через REST-эндпойнт аналитики.

Поддерживается набор стратегий разделения, реализованных в классе BranchSplitStrategies: случайное разделение по весам (random), детерминированное разделение по хешу пользовательского идентификатора с фиксированной солью (hash), разделение по остатку от деления идентификатора на количество вариантов (modulo), разделение по правилам атрибутов входного объекта (attribute), процентная раскатка (percentage), стратифицированное разделение по заданному полю (stratified). Стратегия выбирается в конфигурации узла BranchSplitNodeExecutor. Узел поддерживает два режима — режим split (разделение потока на параллельные ветви для А/В-эксперимента) и режим pick (выбор одной ветви — используется для feature-флагов и управляемой раскатки функциональности на подмножество пользователей без повторного развёртывания). Результат разделения оформляется в виде структуры SplitEnvelope, содержащей идентификатор выбранного варианта, оригинальный вход и метаданные эксперимента; структура передаётся вниз по соответствующей ветви и обеспечивает последующую обработку без потери контекста принадлежности к варианту.

Серверный модуль аналитики размещён в пакете analytics и состоит из четырёх классов. Репозиторий AbAnalyticsRepository выполняет чтение результатов исполнения по конкретному workflow с группировкой по вариантам эксперимента, возвращая количество просмотров и

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

конверсий каждого варианта, а также количество исключённых записей (записи с некорректным или отсутствующим выбором варианта). Указанные операции чтения выполняются в транзакции с уровнем изоляции `@Transactional(readOnly = true)` для обеспечения согласованного чтения данных двух связанных запросов. Сервис `AbAnalyticsService` агрегирует результаты по вариантам, рассчитывает выборочные конверсии, доверительные интервалы на заданном уровне значимости и вызывает статистический тест. Класс `StatTest` реализует z-тест двух пропорций для проверки нулевой гипотезы о равенстве конверсий вариантов и возвращает p-значение разности. Класс `AbAnalyticsDtos` содержит типы данных ответа REST-эндпойнта, включающие просмотры, конверсии, выборочные конверсии и границы доверительных интервалов для каждого варианта, p-значение и индикатор статистической значимости разности.

REST-эндпойнт `GET /v1/workflows/{workflowId}/ab-analytics` реализован в контроллере `AbAnalyticsController` и возвращает сводный отчёт по A/B-эксперименту в формате JSON. Эндпойнт используется клиентским компонентом `analytics-panel` для отображения вкладки продуктовой аналитики в нижней панели редактора с `traffic bars` по вариантам, таблицей конверсий и визуализацией доверительных интервалов средствами библиотеки `Chart.js`.

Корректность модуля подтверждается набором автоматических тестов: модульные тесты стратегий разделения (`BranchSplitStrategiesTest`), модульные тесты узла разделения (`BranchSplitNodeExecutorTest`), модульные тесты обёртки `SplitEnvelopeTest`, модульные тесты узла слияния (`BranchMergeNodeExecutorTest`), интеграционные тесты исполнения `workflow` с разделением и слиянием (`WorkflowExecutionServiceBranchTest`), модульные тесты сервиса аналитики (`AbAnalyticsServiceTest`), модульные тесты статистического теста (`StatTestTest`), интеграционные тесты REST-эндпойнта аналитики (`AbAnalyticsControllerTest`).

В качестве направлений развития предусмотрены: поддержка многоруких бандитов с динамическим перераспределением трафика по мере накопления данных; учёт временной зависимости конверсий с применением `sequential testing`; интеграция с внешними системами продуктовой аналитики через стандартные форматы экспорта.

3.2.10. Стратегия автоматического тестирования и непрерывная интеграция

Серверная часть содержит критически важные алгоритмы (топологическая сортировка, параллельное исполнение, изолированная песочница, статистическое тестирование), сбой в которых способны привести к нарушению функциональных характеристик программного продукта.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Серверная часть должна обеспечиваться многоуровневой стратегией автоматического тестирования с автоматическим запуском в системе непрерывной интеграции и контролем покрытия кода.

Для модульных тестов применяется библиотека JUnit Jupiter версии 5 в сочетании со стандартной библиотекой утверждений. Для интеграционных тестов с реальными контейнерами PostgreSQL и MinIO применяется библиотека Testcontainers версии 1.21.1, обеспечивающая автоматический запуск контейнеров при старте тестового класса и их освобождение по окончании. Для измерения покрытия кода применяется библиотека JaCoCo версии 0.8.13 с интеграцией в Maven через jacoco-maven-plugin. Для системы непрерывной интеграции выбрана платформа GitHub Actions, предоставляющая встроенную интеграцию с репозиторием проекта и поддержку сервисных контейнеров для интеграционных тестов.

Модульные и интеграционные тесты размещены в директории backend/src/test/kotlin/. Состав тестовых классов включает DataflowNodeExecutorsTest (модульные тесты пяти операций обработки потоков данных), MvpIntegrationTests (интеграционные тесты сквозных сценариев), AllaApplicationTests (тесты загрузки контекста Spring), BranchSplitNodeExecutorTest, BranchSplitStrategiesTest, SplitEnvelopeTest, BranchMergeNodeExecutorTest (тесты узлов разделения и слияния ветвей), WorkflowExecutionServiceBranchTest (интеграционные тесты исполнения с разделением и слиянием), AbAnalyticsServiceTest, AbAnalyticsControllerTest, StatTestTest (тесты модуля аналитики), ConnectionSkeletonTest (тесты модели соединений графа), модульные тесты на HttpNodeExecutor, BlobService, TriggerScheduler, WorkflowExecutionService, CanonicalJson, Hashing, WorkflowService.

Пайплайн непрерывной интеграции реализован в файле .github/workflows/ci.yml. На каждый pull request и каждый push в основную ветку поднимаются сервисные контейнеры PostgreSQL и MinIO, выполняется команда `mvn verify`, включающая компиляцию исходного кода, прогон модульных тестов, прогон интеграционных тестов и генерацию отчёта покрытия JaCoCo. Дополнительные workflow coverage-badge.yml, backend.yml и frontend.yml реализуют автоматическое обновление SVG-бейджа покрытия в директории .github/badges/, сборку Docker-образов серверной и клиентской частей и публикацию в GitHub Container Registry под адресами ghcr.io/arklual/cursach-backend и ghcr.io/arklual/cursach-frontend. Развёртывание на сервер выполняется отдельным workflow через SSH-action.

Конфигурация JaCoCo в файле pom.xml включает два этапа: jacoco-prepare-agent на фазе compile (подключение Java-агента для инструментирования байт-кода) и jacoco-report на

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

фазе verify (генерация HTML- и CSV-отчётов в директории `target/site/jacoco/`). Конфигурация дополнена правилом проверки минимального порога покрытия 90% по строкам, обеспечивающим fail-fast поведение CI-пайплайна при регрессии покрытия. Фактическое покрытие серверной части по строкам составляет не менее 90%; покрытие критических пакетов (`run`, `executor`, `blob`) — не менее 95%.

В качестве направлений развития предусмотрены: property-based testing средствами библиотеки Kotest Property для автоматической генерации корректных DAG и проверки инвариантов алгоритмов; нагрузочное тестирование средствами k6 или Gatling для измерения throughput движка исполнения; chaos engineering с программным внедрением случайных сбоев в исполнители узлов; mutation testing средствами PIT для оценки качества тестов.

3.2.11. Схема реляционной базы данных

Серверная часть оперирует множеством взаимосвязанных доменных сущностей: workflow, ревизия workflow, именованная версия workflow, снимок workflow, запуск workflow, узел запуска, триггер, индекс контент-адресуемого хранилища. Серверная часть должна обеспечивать целостность данных через внешние ключи и каскадные правила удаления, индексирование частых запросов, миграционный механизм для эволюции схемы и иммутабельность исторических ревизий и версий.

Для управления миграциями схемы применяется библиотека Liquibase с описанием changeset-ов в формате YAML. Альтернативная библиотека Flyway была отвергнута в пользу Liquibase по причине поддержки YAML-формата с улучшенной читаемостью по сравнению с SQL-only форматом Flyway, наличия богатого набора preCondition и rollback-сценариев, поддержки context-зависимого исполнения changeset для разделения development и production-окружений.

Миграции расположены в директории `backend/src/main/resources/db/changelog/`. Корневой файл `db.changelog-master.yaml` инcludes инкрементальные changeset-ы из поддиректории `changes/`. Реализовано пять changeset-ов. Changeset `001-init.yaml` создаёт базовый набор таблиц, индексы и внешние ключи с правилами каскадного удаления и `ON DELETE RESTRICT` для защиты исторических версий. Changeset `002-trigger-node-id.yaml` добавляет колонку `node_id` в таблицу `triggers` для возможности привязки триггера к конкретному стартовому узлу графа. Changeset `003-restore-after-drop.yaml` реализует защиту от непреднамеренного удаления ключевых таблиц при автоматической пересборке development-окружения. Changeset `004-add-is-demo.yaml` добавляет флаг `is_demo` в таблицу `workflows` для отметки посевных демонстрационных workflow

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

и их фильтрации в клиентской части. Changeset 005-workflow-snapshots.yaml создаёт отдельную таблицу workflow_snapshot для именованных снапшотов уровня документа.

Таблица	Назначение	Ключевые поля и ограничения
workflows	Центральная сущность workflow	id (UUID PK), name, description, current_version_id (FK), is_demo (bool), created_at, updated_at
workflow_revision	Автоматические ревизии конфигурации workflow	id (bigserial PK), workflow_id (FK ON DELETE CASCADE), revision_number (UNIQUE per workflow), graph_skeleton (jsonb), created_at
workflow_version	Именованные версии (snapshots на уровне ревизии)	id (bigserial PK), workflow_id (FK), version_tag, root_revision_id (FK ON DELETE RESTRICT), created_at
workflow_snapshot	Именованные снапшоты уровня документа	id (bigserial PK), workflow_id (FK), revision_id (FK), name, description, created_at
workflow_run	Запуски workflow	id (bigserial PK), workflow_id (FK), workflow_revision_id (FK), status, trigger_type, started_at, finished_at, input (jsonb), output (jsonb)
node_run	Результат исполнения каждого узла	id (bigserial PK), workflow_run_id (FK ON DELETE CASCADE), node_id, node_type, status, config_hash, input_json, output_json, error_message
triggers	Триггеры запуска workflow	id (bigserial PK), workflow_id (FK), type (enum), config (jsonb), token (UNIQUE NULL), enabled (bool), node_id
blob_index	Индекс контент-адресуемого хранилища MinIO	content_hash (text PK, SHA-256), s3_key, size_bytes, created_at

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Таблица 2. Состав основных таблиц схемы

Связи между сущностями схемы реляционной базы данных, направленность каскадных удалений и ограничения целостности приведены на рисунке 3.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

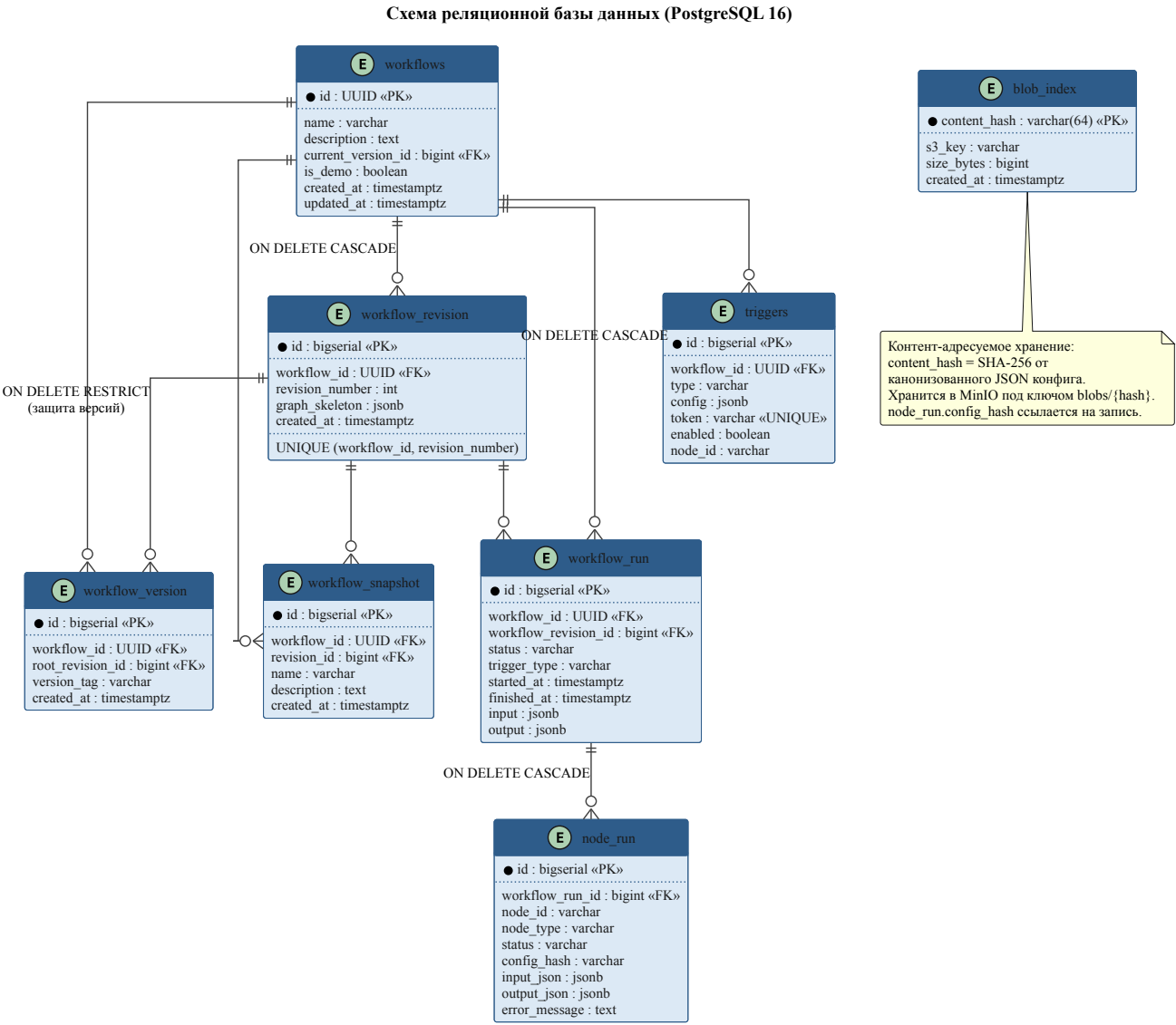


Рис. 3. ER-диаграмма схемы реляционной базы данных

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Состав индексов обеспечивает эффективное выполнение частых запросов. Индекс `workflow_run(workflow_id, started_at DESC)` поддерживает запросы получения истории запусков `workflow`. Индекс `node_run(workflow_run_id)` поддерживает получение всех узлов запуска. Индекс `triggers(workflow_id, enabled)` поддерживает быструю загрузку активных триггеров. Уникальный индекс `triggers(token)` поддерживает поиск триггера по `webhook`-токену.

Правила каскадного удаления настроены с учётом обеспечения целостности и защиты исторических данных. Удаление записи `workflow` каскадно удаляет связанные записи `workflow_revision`, `workflow_run` и `node_run`. Ограничение `ON DELETE RESTRICT` на внешнем ключе `workflow_version.root_revision_id` блокирует удаление ревизии при наличии ссылок из именованных версий.

В качестве направлений развития предусмотрены: партиционирование таблиц `workflow_run` и `node_run` по временным диапазонам поля `started_at` для эффективной обработки многомесячной истории; архивная таблица `workflow_run_archive` с фоновым переносом запусков старше заданного срока; материализованные представления для часто запрашиваемых агрегатов; интеграция с расширением `TimescaleDB` для работы с временными рядами событий.

3.2.12. Программный интерфейс REST API

Клиентская часть программного продукта и внешние информационные системы взаимодействуют с серверной частью через программный интерфейс REST. Серверная часть должна обеспечивать стабильный контракт интерфейса, автоматически генерируемую документацию, синхронизированную с реализацией, единообразную обработку ошибок с машинно-читаемыми кодами, декларативную валидацию входных данных и версионирование интерфейса через префикс пути.

Для документации REST API применяется библиотека `springdoc-openapi` версии 2.7.0, публикующая спецификацию OpenAPI 3.0. Альтернативные варианты — Swagger 2.0, gRPC с `protobuf`-сериализацией и GraphQL — были отвергнуты. Swagger 2.0 является устаревшим форматом и заменён OpenAPI 3.0 в индустрии. gRPC не поддерживается браузерными клиентами без дополнительного проху-слоя и менее удобен для отладки. GraphQL является избыточным для стадии MVP и требует значительной переработки модели данных. Для версионирования интерфейса выбран префикс пути `/v1/` как наиболее простой для отладки через `curl` и `Postman`.

Все REST-контроллеры размещены в пакете `api`. Контроллеры аннотированы `@RestController`, `@RequestMapping` с базовым путём ресурса и `@Tag` для группировки в интерфейсе

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Swagger UI. DTO запросов и ответов размещены в подпакете `api.dto`, аннотированы Jackson-аннотациями для сериализации в JSON в нотации `snake_case`, аннотациями `bean validation` для декларативной валидации и аннотациями OpenAPI для генерации описаний полей в спецификации.

Состав REST-эндпойнтов серверной части приведён в таблице 3.

Метод	Эндпойнт	Назначение
GET	/v1/workflows	Список workflow с пагинацией
POST	/v1/workflows	Создание нового workflow
GET	/v1/workflows/{id}	Получение workflow с актуальной ревизией
PUT	/v1/workflows/{id}	Сохранение конфигурации (создание новой ревизии)
DELETE	/v1/workflows/{id}	Удаление workflow с каскадным удалением связанных данных
GET	/v1/workflows/{id}/versions	Список именованных версий
POST	/v1/workflows/{id}/versions	Создание именованной версии
PUT	/v1/workflow-versions/{vid}/graph	Обновление графа конкретной версии (черновик)
GET	/v1/workflows/{id}/snapshots	Список именованных снапшотов
POST	/v1/workflows/{id}/snapshots	Создание именованного снапшота
DELETE	/v1/workflows/{id}/snapshots/{sid}	Удаление снапшота
POST	/v1/workflows/{id}/runs	Запуск исполнения (ручной триггер)
GET	/v1/workflows/{id}/runs	История запусков workflow
GET	/v1/workflow-runs/{runId}	Состояние запуска и список node_run
GET	/v1/workflow-runs/{runId}/result	Агрегированный результат запуска
GET	/v1/node-runs/{nodeRunId}	Детальный результат одного узла
GET	/v1/workflows/{id}/triggers	Список триггеров
POST	/v1/workflows/{id}/triggers	Создание триггера

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Метод	Эндпойнт	Назначение
PATCH	/v1/workflows/{id}/triggers/{tid}	Обновление триггера
POST	/v1/webhook/{token}	Входная точка webhook-триггера
GET	/v1/workflows/{workflowId}/ab-analytics	Сводный отчёт по А/В-эксперименту

Таблица 3. REST-эндпойнты серверной части

Единообразная обработка ошибок реализована глобальным обработчиком исключений `RestExceptionHandler`, аннотированным `@RestControllerAdvice`. Обработчик преобразует ошибки валидации DTO в HTTP-ответ 400 Bad Request с телом унифицированной структуры, содержащим код ошибки `validation_failed`, человеко-читаемое сообщение и массив описаний нарушений по полям. Исключения ненахождения сущностей преобразуются в HTTP-ответ 404 Not Found с кодом ошибки `not_found`. Исключения нарушения бизнес-инвариантов (например, попытка восстановления из несуществующей версии) преобразуются в HTTP-ответ 409 Conflict с кодом ошибки `conflict`. Прочие необработанные исключения преобразуются в HTTP-ответ 500 Internal Server Error с кодом ошибки `internal_error` и идентификатором корреляции, по которому полный stack trace доступен в журнале серверной части.

Интерактивная документация Swagger UI доступна по адресу `http://localhost:8080/v1/swagger-ui.html`. Спецификация OpenAPI в формате JSON доступна по адресу `http://localhost:8080/v1/v3/api-docs`, в формате YAML — по адресу `http://localhost:8080/v1/v3/api-docs.yaml`. Конфигурация спецификации (заголовок, версия, контактные данные, перечень серверов) определена в классе `OpenApiConfig`.

В качестве направлений развития предусмотрены: автоматическая генерация типизированного клиента TypeScript для клиентской части из OpenAPI-спецификации (npm-скрипт `gen:api` уже добавлен в файл `package.json` клиентской части); ограничение частоты запросов средствами библиотеки `Bucket4j` для защиты webhook-эндпойнтов от злоупотреблений; журнал аудита мутлирующих запросов с записью идентификатора пользователя, тела запроса и результата операции.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.3. Описание и обоснование выбора метода организации входных и выходных данных

3.3.1. Описание метода организации входных данных

Входные данные серверной части поступают из четырёх источников. Первый источник — клиентская часть программного продукта, передающая пользовательские запросы на создание, чтение, обновление и удаление workflow, запуск исполнения, управление триггерами в виде JSON-тел HTTP-запросов через REST API. Второй источник — внешние информационные системы, передающие произвольное JSON-тело через webhook-эндпойнт `POST /v1/webhook/{token}` в качестве входа запускаемого workflow. Третий источник — внутренний планировщик cron-задач, инициирующий запуск scheduler-триггеров без внешних входных данных. Четвёртый источник — внешние сервисы, опрашиваемые HTTP-узлами в процессе исполнения workflow; тела ответов внешних сервисов становятся входами последующих узлов в цепочке.

Метаданные сущностей (имена workflow, описания, статусы запусков, тип триггера) хранятся в реляционной базе данных PostgreSQL в типизированных колонках. Структура графа workflow (узлы, рёбра, типы и хеши конфигураций) хранится в колонках типа jsonb, что обеспечивает гибкость хранения произвольных JSON-документов с одновременной возможностью индексирования по ключам через GIN-индексы при необходимости. Объёмные конфигурации узлов хранятся в объектном хранилище MinIO по контент-адресуемой схеме.

3.3.2. Описание метода организации выходных данных

Выходные данные серверной части передаются по трём каналам. Первый канал — REST API, возвращающий JSON-ответы клиентской части и внешним системам: данные сущностей, списки, статусы операций, сводные показатели А/В-аналитики. Второй канал — WebSocket, транслирующий события жизненного цикла исполнения клиентской части в режиме реального времени. Третий канал — внутреннее хранение результатов исполнения: входы, выходы и сообщения об ошибках каждого узла сохраняются в таблице `node_run` в jsonb-колонках, агрегированный результат workflow сохраняется в таблице `workflow_run` в колонке `output`. Указанные данные доступны клиентской части через REST API для отображения детализированных результатов исторических запусков.

3.3.3. Обоснование выбора метода

Гибридная схема хранения, сочетающая реляционную базу данных PostgreSQL для метаданных и структуры графа в формате jsonb с объектным хранилищем MinIO для объёмных конфигураций узлов в контент-адресуемой схеме, оптимальна для специфики программного

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

продукта. Метаданные сущностей требуют гарантий ACID и индексирования, обеспечиваемых реляционной базой данных. Объёмный содержательный контент требует эффективного дедуплицированного хранилища, обеспечиваемого объектным хранилищем с контент-адресуемой схемой. Канал WebSocket для трансляции событий исполнения исключает необходимость периодического опроса со стороны клиентской части, что снижает нагрузку на серверную часть и обеспечивает требуемую латентность обновления интерфейса. REST API в качестве основного программного интерфейса соответствует индустриальному стандарту и обеспечивает совместимость с произвольными HTTP-клиентами.

3.4. Требования к составу и параметрам технических средств

Минимальные требования к техническим средствам для развёртывания серверной части программного продукта: процессор не менее 4 ядер архитектуры x86_64; оперативная память не менее 8 ГБ (распределение: 4 ГБ для JVM, 2 ГБ для PostgreSQL, 1 ГБ для MinIO, 1 ГБ для docker daemon и контейнеров изолированной песочницы); дисковое пространство не менее 20 ГБ. Требования к программным средствам: JDK 24 для запуска fat-JAR, PostgreSQL 16, MinIO стабильной версии, Docker Engine 20.10 или совместимый. Все программные компоненты допускают развёртывание в Docker-контейнерах через предоставленный файл `deploy/docker-compose.prod.yml`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ КАЧЕСТВЕННЫЕ И КОЛИЧЕСТВЕННЫЕ ПОКАЗАТЕЛИ

Целевые качественные и количественные показатели серверной части программного продукта приведены ниже. 95-й перцентиль времени отклика REST API на запросы чтения при типовой нагрузке (10 одновременных подключений) не превышает 500 мс. Время от приёма триггера до перехода первого узла workflow в статус «running» не превышает 5 секунд с учётом задержки запуска контейнера изолированной песочницы и инициализации цепочки CompletableFuture. Параллелизм исполнения обеспечивается пулом потоков размера 16. Покрытие исходного кода серверной части по строкам, измеряемое библиотекой JaCoCo, составляет не менее 90%; покрытие критических пакетов (run, executor, blob) — не менее 95%.

Количество автоматических тестов серверной части: модульные тесты на JUnit Jupiter — около 55, интеграционные тесты на Testcontainers — около 12, доля успешно проходящих тестов на ветке main — 100%. Изоляция пользовательского кода в узлах Code Node обеспечивается одноразовым Docker-контейнером с параметрами `--network none, --memory 256m, --cpus 1, --read-only, --tmpfs /tmp:rw,size=16m, --cap-drop ALL, --security-opt no-new-privileges, --pids-limit 64`, тайм-аут исполнения 5 секунд плюс служебный overhead 30 секунд. Автоматическая документация REST API представлена спецификацией OpenAPI 3.0 и интерактивным интерфейсом Swagger UI, синхронизированными с реализацией. Время полного прогона пайплайна непрерывной интеграции на pull request не превышает 12 минут. Готовность серверной части после команды `docker compose up -d` достигается за время, не превышающее 60 секунд (после первой загрузки Docker-образов).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Boot Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> (дата обращения: 05.12.2025).
2. Kotlin Language Documentation [Электронный ресурс]. — URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 08.12.2025).
3. PostgreSQL 16 Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 12.12.2025).
4. Docker Engine Documentation [Электронный ресурс]. — URL: <https://docs.docker.com/engine/> (дата обращения: 18.03.2026).
5. MinIO Object Storage Documentation [Электронный ресурс]. — URL: <https://min.io/docs/minio/linux/index.html> (дата обращения: 20.02.2026).
6. Liquibase Documentation [Электронный ресурс]. — URL: <https://docs.liquibase.com/> (дата обращения: 15.12.2025).
7. springdoc-openapi Documentation [Электронный ресурс]. — URL: <https://springdoc.org/> (дата обращения: 05.02.2026).
8. OpenAPI Specification 3.0.3 [Электронный ресурс]. — URL: <https://spec.openapis.org/oas/v3.0.3> (дата обращения: 18.12.2025).
9. Java CompletableFuture API [Электронный ресурс]. — URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/CompletableFuture.html> (дата обращения: 10.02.2026).
10. Testcontainers for Java [Электронный ресурс]. — URL: <https://java.testcontainers.org/> (дата обращения: 12.04.2026).
11. JaCoCo Code Coverage Library [Электронный ресурс]. — URL: <https://www.jacoco.org/jacoco/trunk/doc/> (дата обращения: 15.04.2026).
12. Клеппман М. Высоконагруженные приложения // Программирование, масштабирование, поддержка. Питер. – 2018.
13. Ньюмен С. Создание микросервисов. 2-е издание. – Питер, 2023.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 81 03-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]